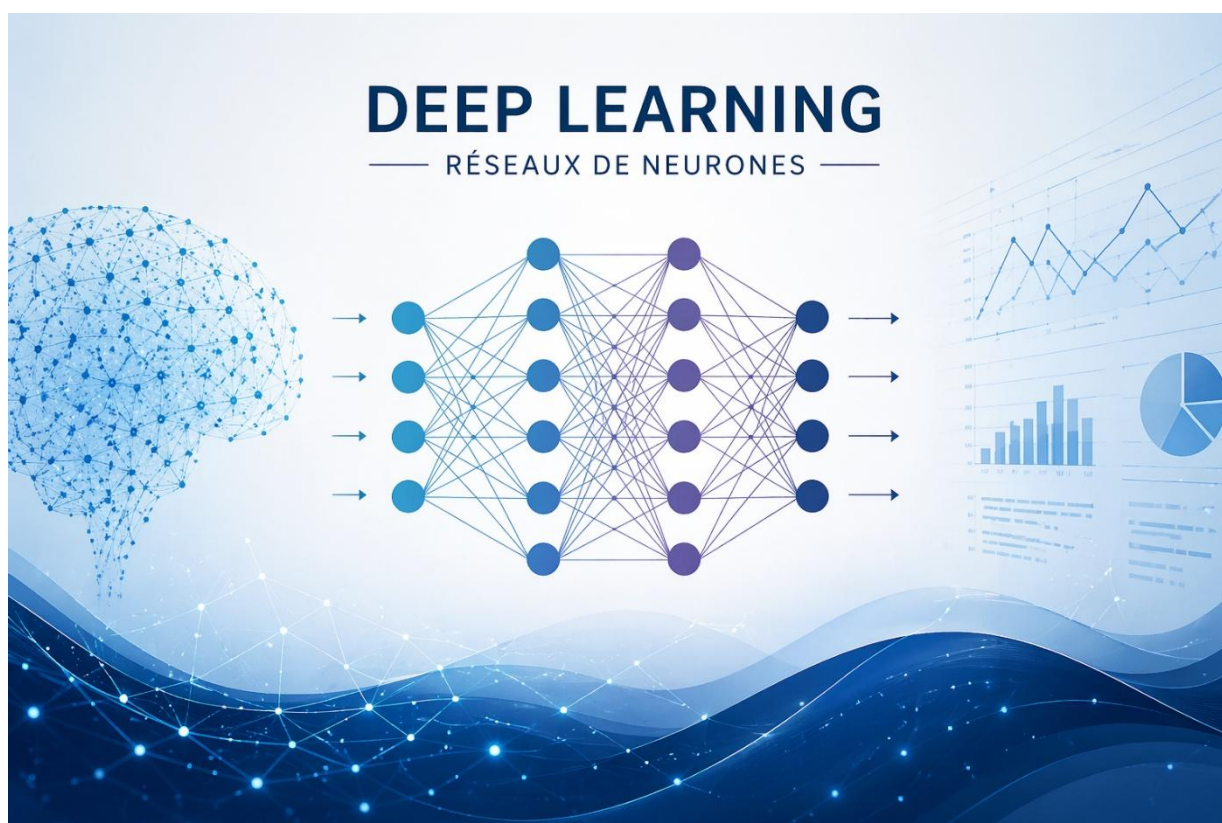


Régularisation et Hyperparamètres dans les Réseaux de Neurones

Compte-rendu complet avec explications, schémas et exemples de code



Encadré par : Pr. M. CHERRADI

Réalisé par : Nada EL-KHAMLICHI

Module : Deep Learning

Année universitaire : 2025 / 2026

1. Introduction générale :

Avec l'essor du **Deep Learning**, les réseaux de neurones artificiels se sont imposés comme des outils incontournables pour résoudre des problèmes complexes dans divers domaines tels que la vision par ordinateur, le traitement du langage naturel et l'analyse de données massives. Leur capacité à apprendre automatiquement des représentations à partir des données leur confère une grande puissance, mais également une certaine complexité dans leur conception et leur mise en œuvre.

En effet, la performance d'un réseau de neurones ne dépend pas uniquement des données utilisées ou de l'algorithme choisi, mais aussi d'un ensemble de choix effectués avant et pendant l'entraînement du modèle. Ces choix influencent directement la qualité de l'apprentissage, la rapidité de convergence ainsi que la capacité du modèle à généraliser sur de nouvelles données.

Dans ce contexte, il devient essentiel de comprendre les différents éléments qui interviennent dans le processus d'apprentissage d'un réseau de neurones, ainsi que leur impact sur les performances finales du modèle. Une bonne maîtrise de ces éléments permet de concevoir des modèles plus robustes, plus efficaces et mieux adaptés aux problèmes étudiés.

Ce travail s'inscrit dans cette perspective et vise à présenter de manière structurée les concepts fondamentaux liés à la configuration et à l'optimisation des réseaux de neurones, en mettant en évidence leur rôle dans l'amélioration des performances.

2. Définition des hyperparamètres :

Dans le domaine du **Machine Learning** et plus particulièrement du **Deep Learning**, les hyperparamètres désignent un ensemble de variables définies **avant le début de l'entraînement** d'un modèle. Contrairement aux paramètres internes du réseau, ces valeurs ne sont pas apprises automatiquement à partir des données, mais sont fixées par le concepteur du modèle.

Les hyperparamètres jouent un rôle essentiel dans le processus d'apprentissage, car ils influencent directement le comportement du modèle, notamment la manière dont il apprend, la vitesse de convergence ainsi que sa capacité à généraliser.

3. Différence entre paramètres et hyperparamètres :

Les **paramètres** sont les variables internes du modèle, telles que les **poids (weights)** et les **biais (bias)**. Ils sont ajustés automatiquement au cours du processus d'entraînement à l'aide d'algorithmes d'optimisation, dans le but de minimiser la fonction de coût.

En revanche, les **hyperparamètres** ne sont pas appris par le modèle. Ils sont définis en amont et contrôlent le processus d'apprentissage. Ils influencent la manière dont les paramètres sont mis à jour, mais ne sont pas eux-mêmes modifiés durant l'entraînement.

4. Importance des hyperparamètres :

Les hyperparamètres jouent un rôle déterminant dans la performance globale d'un modèle de Deep Learning. Un mauvais choix de ces paramètres peut entraîner une dégradation significative des résultats, même si les données utilisées sont de bonne qualité.

En effet, les hyperparamètres influencent plusieurs aspects clés de l'apprentissage, notamment :

- la **vitesse de convergence** du modèle,
- la **stabilité de l'entraînement**,
- ainsi que la **capacité de généralisation** sur de nouvelles données.

Un réglage inadéquat peut conduire à deux problèmes majeurs :

- le **surapprentissage (overfitting)**, où le modèle mémorise les données d'entraînement sans parvenir à généraliser,
- le **sous-apprentissage (underfitting)**, où le modèle est incapable de capturer les relations présentes dans les données.

Par conséquent, l'ajustement des hyperparamètres, appelé **hyperparameter tuning**, est une étape essentielle pour obtenir un modèle performant et robuste. Cette étape consiste à tester différentes configurations afin de trouver celles qui offrent les meilleurs résultats.

5. Types des hyperparamètres :

Les hyperparamètres d'un réseau de neurones peuvent être classés en plusieurs catégories selon leur rôle dans le processus d'apprentissage. De manière générale, on distingue principalement deux **grandes** familles : les **hyperparamètres d'optimisation** et les **hyperparamètres d'architecture**. Cette classification permet de mieux comprendre leur influence sur le comportement et les performances du modèle.

5.1 Hyperparamètres d'optimisation :

Les hyperparamètres d'optimisation sont liés à la manière dont le réseau apprend à partir des données. Ils contrôlent le processus d'entraînement et influencent directement la convergence du modèle, la stabilité de l'apprentissage et la qualité de la solution obtenue.

Parmi les hyperparamètres d'optimisation les plus connus, on trouve :

- **le taux d'apprentissage (learning rate) :**

Il contrôle la taille du pas lors de la mise à jour des poids durant la descente de gradient.

Règle de mise à jour des poids :

$$W_{\text{nouveau}} = W_{\text{ancien}} - \alpha * dL/dW$$

Effets du taux d'apprentissage :

Valeur du taux d'apprentissage (α)	Comportement	Conséquence
Trop élevé (0.1 – 1)	Oscillations importantes, divergence du processus d'apprentissage	Le modèle ne converge pas
Optimal (0.001 – 0.01)	Convergence stable et régulière	Bonne performance du modèle
Trop faible (0.00001)	Apprentissage très lent	Temps d'entraînement excessif

Exemple de code Python (PyTorch) :

```
import torch.optim as optim

# Définir l'optimiseur avec un taux d'apprentissage
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Scheduler : réduire lr si la loss stagne
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
                                                  patience=5, factor=0.5)
```

📌 *Technique avancée : Le Learning Rate Scheduling (décroissance progressive de α) améliore souvent la convergence finale*

- **la taille du lot (*batch size*) :**

Il correspond au nombre d'exemples traités avant chaque mise à jour .

- Petit batch (ex: 16–32) → gradient bruité → meilleure généralisation, mais plus lent
- Grand batch (ex: 256–512) → gradient précis → convergence stable, mais risque d'overfitting
- Full batch → gradient exact mais très coûteux en mémoire

- **le nombre d'époques (*epochs*) :**

Il représente le nombre de passages complets sur l'ensemble des données d'entraînement .

Code Python — boucle d'entraînement standard :

```
batch_size = 32
num_epochs = 100

for epoch in range(num_epochs):
    for X_batch, y_batch in dataloader: # batch_size=32
        optimizer.zero_grad()
        predictions = model(X_batch)
        loss = criterion(predictions, y_batch)
        loss.backward()
        optimizer.step()
```

- **le choix de l'optimiseur :**

Le choix de l'algorithme d'optimisation impacte directement la vitesse de convergence et la qualité finale du modèle.

Parmi les optimiseurs courants :

- **SGD (Stochastic Gradient Descent)** repose sur la mise à jour des poids à partir de mini-batches de données. Il s'agit d'une méthode simple et peu coûteuse en mémoire. Toutefois, il reste sensible au choix du taux d'apprentissage et peut présenter une convergence relativement lente.
- **Momentum** constitue une amélioration de SGD en introduisant un terme de vitesse basé sur l'accumulation des gradients passés. Cette approche permet de réduire les oscillations et d'accélérer la convergence. En revanche, elle nécessite l'introduction d'un hyperparamètre supplémentaire, généralement noté β .
- **Adam (Adaptive Moment Estimation)** combine les avantages du momentum et de l'adaptation du taux d'apprentissage en utilisant les moments d'ordre un et deux des gradients. Il est reconnu pour sa rapidité et sa robustesse, ce qui en fait l'un des optimiseurs les plus utilisés en pratique. Cependant, il est plus gourmand en mémoire et peut parfois offrir une capacité de généralisation inférieure.
- **RMSprop** ajuste dynamiquement le taux d'apprentissage en fonction de la variance des gradients. Cette méthode est particulièrement efficace pour les données séquentielles, notamment dans les réseaux récurrents. Néanmoins, elle ne prend pas en compte le mécanisme classique de momentum.

Optimiseur	Formules de mise à jour	Description
SGD (Stochastic Gradient Descent)	$w_{t+1} = w_t - \eta \nabla L(w_t)$	Descente de gradient stochastique classique. η : learning rate
Momentum	$v_{t+1} = \beta v_t - \eta \nabla L(w_t)$ $w_{t+1} = w_t + v_{t+1}$	Accumule la "vitesse" v pour réduire les oscillations. β : coefficient de momentum
RMSprop	$s_{t+1} = \beta s_t + (1 - \beta)(\nabla L(w_t))^2$ $w_{t+1} = w_t - \frac{\eta}{\sqrt{s_{t+1}} + \epsilon} \nabla L(w_t)$	Adapte le learning rate selon la moyenne exponentielle des grads au carré. ϵ : petite constante (ex: 10^{-8})
Adam	$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \nabla L(w_t)$ $v_{t+1} = \beta_2 v_t + (1 - \beta_2) (\nabla L(w_t))^2$ $\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}$ $\hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}$ $w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}} + \epsilon} \hat{m}_{t+1}$	Combine Momentum (1er ordre) et RMSprop (2e ordre). β_1, β_2 : coefficients (ex: 0.9, 0.999) t : pas de temps

5.2 Hyper-paramètres d'Architecture :

- **Profondeur et Largeur du Réseau :**

Nombre de couches (profondeur) : détermine la complexité des représentations apprises.

- Réseau peu profond (1–2 couches cachées) → bon pour problèmes simples
- Réseau profond (10+ couches) → capture des features hiérarchiques complexes

Nombre de neurones par couche (largeur) : contrôle la capacité de représentation à chaque niveau.

Code Python — définition d'un MLP (Multi-Layer Perceptron) :

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        layers = []
        prev_dim = input_dim
        for h in hidden_dims:
            layers.append(nn.Linear(prev_dim, h))
            layers.append(nn.ReLU())
            prev_dim = h
        layers.append(nn.Linear(prev_dim, output_dim))
        self.net = nn.Sequential(*layers)

model = MLP(input_dim=784, hidden_dims=[256,128,64], output_dim=10)
```

- **Fonctions d'Activation :**

Les fonctions d'activation jouent un rôle essentiel dans les réseaux de neurones, car elles introduisent de la **non-linéarité**, permettant au modèle de capturer des relations complexes dans les données. Plusieurs fonctions sont couramment utilisées, chacune adaptée à des contextes spécifiques.

La fonction **Sigmoid** transforme les valeurs d'entrée dans l'intervalle $[0,1]$, ce qui la rend particulièrement adaptée aux problèmes de **classification binaire**. Cependant, elle peut souffrir du problème du gradient qui disparaît.

La fonction **Tanh** produit des valeurs comprises entre $[-1,1]$. Elle est souvent utilisée dans les **couches cachées des réseaux récurrents (RNN)**, car elle centre les données autour de zéro, ce qui facilite l'apprentissage.

La fonction **ReLU (Rectified Linear Unit)** retourne zéro pour les valeurs négatives et la valeur elle-même pour les valeurs positives. Elle est largement utilisée dans les **réseaux convolutifs (CNN)** grâce à sa simplicité et son efficacité, notamment pour accélérer l'apprentissage.

La variante **Leaky ReLU** permet de corriger un défaut de ReLU en autorisant une petite pente pour les valeurs négatives (généralement $\alpha=0.01$). Cela permet d'éviter le problème des **neurones morts**, qui ne s'activent plus.

Enfin, la fonction **Softmax** est utilisée en sortie des modèles de **classification multiclasse**. Elle transforme les scores en probabilités dont la somme est égale à 1, facilitant ainsi l'interprétation des résultats.

Code Python — comparaison des activations :

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        layers = []
        prev_dim = input_dim
        for h in hidden_dims:
            layers.append(nn.Linear(prev_dim, h))
            layers.append(nn.ReLU())
            prev_dim = h
        layers.append(nn.Linear(prev_dim, output_dim))
        self.net = nn.Sequential(*layers)

model = MLP(input_dim=784, hidden_dims=[256, 128, 64], output_dim=10)
```

- **Types de Couches :**

Le type de couche définit comment les informations sont traitées et agrégées.

- FC (Fully Connected) : chaque neurone connecté à tous les neurones précédents → utilisé pour la classification finale
- CNN (Convolutional Neural Network) : filtres locaux glissants → excellent pour les images
- RNN (Recurrent Neural Network) : mémoire des états passés → séquences temporelles
- LSTM (Long Short-Term Memory) : portes d'oubli, d'entrée et de sortie → séquences longues
- GRU (Gated Recurrent Unit) : version simplifiée du LSTM → plus rapide à entraîner.

Code Python — couche CNN :

```
# Paramètres clés d'une couche CNN
conv = nn.Conv2d(
    in_channels=3,          # RGB
    out_channels=64,        # nombre de filtres
    kernel_size=3,          # taille filtre 3x3
    stride=1,               # pas du filtre
    padding=1               # garder dimensions
)

# Couche de pooling
pool = nn.MaxPool2d(kernel_size=2, stride=2)
```

- **Dropout :**

Le Dropout est une technique de régularisation architecturale qui désactive aléatoirement des neurones pendant l'entraînement.

Principe : à chaque forward pass, chaque neurone est désactivé avec une probabilité p (taux de dropout).

- Taux typique : $p = 0.2$ à 0.5 pour les couches cachées
- $p = 0$: pas de dropout (risque d'overfitting)
- $p = 0.5$: souvent optimal en pratique
- Le dropout est désactivé automatiquement en mode évaluation (`model.eval()`)

Code Python — intégration du Dropout :

```
class DropoutNet(nn.Module):
    def __init__(self, dropout_rate=0.3):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.dropout = nn.Dropout(p=dropout_rate)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.dropout(x)  # désactivé en eval()
        return self.fc2(x)
```

6. les techniques de régularisation:

La régularisation est un ensemble de techniques visant à réduire l'overfitting (sur-apprentissage) en pénalisant la complexité du modèle.

Problème de l'overfitting : le modèle mémorise les données d'entraînement mais généralise mal sur de nouvelles données.

6.1 Régularisation L1 (Lasso) :

La régularisation L1 ajoute la somme des valeurs absolues des poids à la fonction de coût.

Fonction de coût avec L1 :

```
L_total = L_original + λ × Σ|w_i|
```

- Effet : favorise la parcimonie (sparse weights), certains poids deviennent exactement 0
- Utilité : sélection automatique de features
- Paramètre λ : contrôle l'intensité de la pénalité

6.2 Régularisation L2 (Ridge / Weight Decay) :

La régularisation L2 ajoute la somme des carrés des poids à la fonction de coût.

Fonction de coût avec L2 :

$$L_{\text{total}} = L_{\text{original}} + \lambda \times \sum w_i^2$$

Comparaison L1 vs L2 :

Critère	L1 (Lasso)	L2 (Ridge)
Formule de pénalité	$(\lambda \sum w_i)$	w_i^2
Effet sur les poids	Certains poids deviennent exactement nuls (parcimonie)	Les poids sont réduits mais restent non nuls
Sélection de variables (features)	Oui (sélection automatique)	Non
Différentiabilité	Non différentiable en 0	Différentiable partout
Usage typique	Données à haute dimension, modèles parcimonieux	Cas général, Deep Learning

Code Python — implémentation L1 et L2 :

```
# L2 via weight_decay dans l'optimiseur
optimizer = optim.Adam(model.parameters(),
                        lr=0.001, weight_decay=1e-4)

# L1 manuellement dans la boucle de training
lambda_l1 = 1e-5
l1_penalty = sum(p.abs().sum() for p in model.parameters())
loss = criterion(output, target) + lambda_l1 * l1_penalty

# L1 + L2 (Elastic Net)
lambda_l1, lambda_l2 = 1e-5, 1e-4
l1 = sum(p.abs().sum() for p in model.parameters())
l2 = sum(p.pow(2).sum() for p in model.parameters())
loss = criterion(output, target) + lambda_l1*l1 + lambda_l2*l2
```

6.3 Batch Normalization :

La normalisation par batch normalise les activations de chaque couche pour avoir une moyenne nulle et une variance unitaire.

Formule de normalisation :

$$\hat{x} = (x - \mu_{\text{batch}}) / \sqrt{(\sigma^2_{\text{batch}} + \epsilon)}$$

$$y = \gamma \times \hat{x} + \beta \quad \# \gamma \text{ et } \beta \text{ sont apprenables}$$

- Réduit le problème de covariate shift interne
- Permet des taux d'apprentissage plus élevés
- Agit comme régulariseur (remplace souvent le dropout)

Code Python :

```
class BatchNormNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.bn1 = nn.BatchNorm1d(256) # après fc
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = F.relu(self.bn1(self.fc1(x)))
        return self.fc2(x)
```

7. Architectures Spécifiques et Transfer Learning:

7.1 Architectures CNN célèbres :

Les réseaux de neurones convolutifs ont évolué grâce à plusieurs architectures de référence ayant marqué le domaine de la vision par ordinateur.

AlexNet (2012) a introduit l'utilisation des GPU, de la fonction ReLU et du dropout, ouvrant la voie aux réseaux profonds.

VGG-16 (2014) se distingue par une architecture plus profonde et l'utilisation de filtres 3×3 uniformes.

ResNet-50 (2015) apporte les connexions résiduelles, facilitant l'entraînement des réseaux très profonds.

Enfin, **EfficientNet (2019)** propose une mise à l'échelle optimisée du réseau, améliorant les performances tout en réduisant le coût computationnel.

7.2 Transfer Learning :

C est utiliser un modèle pré-entraîné sur un grand dataset (ex: ImageNet) et l'adapter à une tâche spécifique.

- Feature extraction : geler toutes les couches, entraîner seulement le classifieur final
- Fine-tuning : dégeler progressivement les dernières couches
- Avantage majeur : nécessite beaucoup moins de données

Code Python — Transfer Learning avec ResNet :

```

import torchvision.models as models

# Charger ResNet50 pré-entraîné sur ImageNet
model = models.resnet50(pretrained=True)

# Option 1 : Feature extraction (geler tout)
for param in model.parameters():
    param.requires_grad = False

# Remplacer le classifieur final
num_classes = 10
model.fc = nn.Linear(model.fc.in_features, num_classes)

# Option 2 : Fine-tuning des dernières couches
for param in model.layer4.parameters():
    param.requires_grad = True

```

8. Recherche des Hyper-paramètres Optimaux :

8.1 Grid Search :

Exploration exhaustive de toutes les combinaisons possibles de valeurs d'hyper-paramètres.

- Avantage : explore toutes les combinaisons, résultats reproductibles
- Inconvénient : coûteux en calcul (exponentiel en nombre de paramètres)

Code Python

```

from sklearn.model_selection import GridSearchCV

param_grid = {
    'lr': [0.001, 0.01, 0.1],
    'batch_size': [16, 32, 64],
    'dropout': [0.2, 0.3, 0.5]
}

# 27 combinaisons à évaluer (3×3×3)

```

8.2 Random Search et Bayesian Optimization :

Alternatives plus efficaces au Grid Search pour les espaces d'hyper-paramètres larges.

- Random Search : tire aléatoirement des combinaisons → souvent aussi bon que Grid Search en moins de temps
- Bayesian Optimization (Optuna, Ray Tune) : utilise les résultats passés pour guider la recherche

Code Python avec Optuna :

```
import optuna

def objective(trial): 1 usage
    lr = trial.suggest_float('lr', 1e-5, 1e-1, log=True)
    dropout = trial.suggest_float('dropout', 0.1, 0.5)
    batch_size = trial.suggest_categorical('bs', [16,32,64])
    # ... entraîner le modèle et retourner la val_accuracy
    return val_accuracy

|
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100)
```

9. Conclusion :

Au terme de cette étude, il apparaît que les hyperparamètres jouent un rôle central dans la performance des réseaux de neurones. Bien qu'ils ne soient pas appris automatiquement comme les paramètres du modèle, leur choix influence directement la qualité de l'apprentissage, la vitesse de convergence ainsi que la capacité de généralisation.

Nous avons mis en évidence deux grandes catégories d'hyperparamètres : ceux liés à l'optimisation, qui déterminent la manière dont le modèle apprend, et ceux liés à l'architecture, qui définissent sa structure et sa capacité de représentation. Par ailleurs, les techniques de régularisation telles que L1, L2, le dropout ou encore la batch normalization permettent de limiter le surapprentissage et d'améliorer la robustesse des modèles.

L'étude des différentes architectures, notamment les CNN modernes et les approches de transfer learning, montre également l'importance des choix de conception dans la réussite d'un modèle. De plus, les méthodes de recherche d'hyperparamètres, telles que le Grid Search, le Random Search ou l'optimisation bayésienne, constituent des outils essentiels pour identifier les configurations optimales.

En conclusion, la conception d'un modèle performant ne repose pas uniquement sur les données ou les algorithmes utilisés, mais sur un **équilibre judicieux entre architecture, optimisation et régularisation**. La maîtrise des hyperparamètres apparaît ainsi comme une compétence fondamentale en Deep Learning, indispensable pour développer des modèles efficaces et adaptés à des problèmes réels.